A Dockerfile is an automated build manifest containing step-by-step instructions that assemble a self-contained, reproducible virtual environment (called a Docker Container). In artificial intelligence and computer vision applications, Python code frequently depends on low-level operating system C++ binaries, graphics drivers (e.g., libgl1), and specific versions of numerical libraries. Without containerization, moving code to a different machine often produces missing shared object errors, version conflicts, or crashing dependencies.

The project Dockerfile is structured into six deliberate, hardened stages:

Stage 1: Base Operating System

Directive: FROM python:3.10-slim

Explanation: Instead of a bloated 5-10 GB full operating system, this pulls an official lightweight (~150 MB) Debian Linux system with Python 3.10 pre-installed. It minimizes memory consumption and reduces security vulnerabilities.

Stage 2: Low-Level System Dependencies

Directive: RUN apt-get update && apt-get install -y --no-install-recommends libgl1 libglib2.0-0 curl && rm -rf /var/lib/apt/lists/*

Explanation: OpenCV requires Linux shared C++ libraries (libgl1 and libglib) to decode and manipulate image matrices. Installing these via apt-get ensures cv2 will never crash with missing shared object errors.

Stage 3: Working Directory Definition

Directive: WORKDIR /app

Explanation: Sets the primary execution and storage folder inside the container to /app.

Stage 4: Dependency Caching & Installation

Directive: COPY requirements.txt . followed by RUN pip install --no-cache-dir -r requirements.txt

Explanation: Copies only requirements.txt first and runs pip. This leverages Docker layer caching: if dependencies haven't changed, Docker reuses the cached layer, making future builds near-instantaneous.

Stage 5: Application & Model Ingestion

Directive: COPY models/ ./models/ | COPY src/ ./src/ | COPY tests/test_images/ ./tests/test_images/

Explanation: Copies the trained ResNet-10 deep learning neural network weights, the Python application logic (app.py, detector.py, utils.py), and sample test images into the container.

Stage 6: Port Exposure & Launch Entrypoint

Directive: EXPOSE 8501 | ENTRYPOINT ["streamlit", "run", "src/app.py", "--server.port=8501", "--server.address=0.0.0.0"]

To eliminate the need for memorizing long terminal execution flags, docker-compose.yml stores all operational settings:

• Port Forwarding: Maps host machine port 8501 to container port 8501 ('8501:8501').

• Container Naming: Assigns a clean, human-readable name ('facemask_app').

• Restart Policy: Set to 'unless-stopped' so the container automatically recovers if the system reboots.

Follow these exact steps to launch, inspect, and verify the container on your system:

Step 4.1: Launch Docker Desktop

• Open Docker Desktop from the Windows Start Menu.

• Verify the bottom-left status indicator turns GREEN and displays 'Engine running'.

Step 4.2: Execute the Launch Command in Command Prompt

• Open Command Prompt (cmd) or PowerShell.

• Navigate to the project root: cd C:\Pankaja\AUTOMATED_SDLC_AGENT_PROJECTS\faceMask_detection

• Execute the build and run command:

docker-compose up -d --build

• Docker will download the base layers, install OpenCV, copy models, and report: 'Container facemask_app Started'.

Step 4.3: Terminal Status Verification

- **In Command Prompt, run: docker ps**
- **Confirm the output shows:**

  - IMAGE: facemask-detection:1.0.0

  - STATUS: Up X seconds / minutes

  - PORTS: 0.0.0.0:8501->8501/tcp

  - NAMES: facemask_app

Step 4.4: Web Browser Functional Verification

- **Open Google Chrome, Microsoft Edge, or Firefox.**
- **Navigate to: http://localhost:8501**
- **The web dashboard will load immediately.**
- **Functional Tests to Execute:**

1. Click 'Two Nurses (Putting on Gloves)' -> Verify two faces are detected with green bounding boxes and 'Mask: 95.0%'.

2. Click 'Side Profile (Black Fabric Mask)' -> Verify the angled face is detected with a green box.

3. Click 'Unmasked Portrait' -> Verify a red bounding box appears with 'No Mask'.

4. Click 'Download Annotated Image' -> Verify the .jpg file downloads successfully to your machine.

Step 4.5: Internal Automated Test Execution Inside Docker

- **To run the automated 25-point pytest suite directly inside the Linux container, execute:**

docker exec -it facemask_app pytest tests/

- **Start the container in background: docker-compose up -d**
- **Rebuild and start after code updates: docker-compose up -d --build**
- **View real-time container logs: docker logs -f facemask_app**
- **Stop the container gracefully: docker-compose down**
- Inspect container resource usage (CPU/RAM): docker stats facemask_app